

ADVANCED ENCRYPTION STANDARD (AES-128-bit)

in Verilog

Designed on Xilinx Vivado

Project Type	Hardware Design — Digital VLSI / FPGA
Standard	NIST FIPS 197 — Advanced Encryption Standard (AES-128)
HDL Used	Verilog HDL
Tool	Xilinx Vivado Design Suite
Key Length	128 bits
Data Block Size	128 bits
Number of Rounds	10 (Encryption) + 10 (Decryption)
Architecture	Fully Unrolled Combinational + Sequential Hybrid (Dataflow basically)
Modules Designed	16 Verilog Modules (Encryption + Decryption + Keys)
Author	Vipul Sharma
GitHub	github.com/vipul3112/AES-128-bit

1. Project Overview

1.1 What is AES-128?

The Advanced Encryption Standard (AES) is a symmetric block cipher standardized by the U.S. National Institute of Standards and Technology (NIST) in 2001 under FIPS Publication 197. It replaced the older DES (Data Encryption Standard) and remains the most widely used encryption algorithm in the world today.

AES operates on fixed-size blocks of 128 bits (16 bytes) of plaintext and produces an equally-sized ciphertext block. In AES-128, the secret key is also 128 bits long. The same key is used for both encryption and decryption — making AES a symmetric cipher. AES processes data in a 4×4 matrix of bytes called the state matrix, running it through a sequence of mathematical transformations across multiple rounds to produce ciphertext that is computationally infeasible to reverse without the key.

1.2 Why Implement AES in Hardware?

While AES can be implemented in software (C, Python, etc.), a hardware implementation — realized in Verilog and deployed on an FPGA or synthesized into an ASIC — delivers significant advantages:

- **Speed:** Hardware implementations can process data 10–100× faster than software equivalents. A fully unrolled combinational design can achieve multi-Gbps throughput.
- **Deterministic Latency:** Unlike software running on a CPU with unpredictable OS scheduling, hardware encryption delivers fixed, predictable latency — critical for real-time systems.
- **Isolation:** A dedicated hardware security module (HSM) keeps cryptographic keys isolated from the main processor, reducing attack surface.
- **Power Efficiency:** Custom hardware consumes far less power per encrypted byte compared to general-purpose CPUs, essential for IoT and battery-powered devices.
- **Parallelism:** Hardware allows all 16 S-Box lookups and all four MixColumns operations to execute in parallel — something a sequential CPU cannot do natively.

1.3 Where Are AES Hardware Accelerators Used?

- **Secure Microcontrollers & Smart Cards:** Banking chips (EMV cards), SIM cards, and secure elements embed AES hardware cores.
- **IoT Devices:** Low-power embedded sensors (LoRa, Zigbee, BLE devices) use hardware AES to secure data without draining batteries.
- **Wireless Communication:** WPA2/WPA3 (Wi-Fi), 4G/5G, and Bluetooth Low Energy all mandate AES encryption handled by dedicated hardware blocks.
- **Storage Encryption:** SSDs and eMMC chips implement AES-128/256 in hardware for transparent full-disk encryption (FDE).
- **FPGAs in Defense & Aerospace:** High-speed secure data links in radar, satellite communications, and avionics use FPGA-based AES accelerators.
- **VPN & Network Equipment:** Routers and firewalls offload AES-GCM encryption to dedicated hardware engines for line-rate encryption.

2. Design Architecture

2.1 AES-128 State Matrix & Data Representation

AES treats the 128-bit input as a 4×4 matrix of bytes arranged column-major. Each byte occupies 8 bits, and the 16 bytes are labeled byte[0] through byte[15]. In this Verilog implementation, the 128-bit bus is sliced into 16 individual 8-bit wires (e.g., in[127:120] = byte 0, in[119:112] = byte 1, ..., in[7:0] = byte 15), which matches the AES column-major ordering. This consistent slicing pattern appears identically across SubBytes, ShiftRows, MixColumns, and all other modules.

2.2 Top-Level Module: AES

File: aes_final.v | Module: AES

This is the top-level wrapper that ties both encryption and decryption datapaths together and selects the output based on a 1-bit mode control signal.

```
module AES(out, in, key, mode, Clk);
    input  [127:0] in    - 128-bit plaintext (for encrypt) or ciphertext (fed
to decrypt)
    input  [127:0] key   - 128-bit secret key
    input                mode - 0 = Encrypt, 1 = Decrypt
    input                Clk  - System clock
    output reg [127:0] out - 128-bit result
```

Design Decision: Both the Encryption and Decryption modules run simultaneously and in parallel. The mode signal simply selects which result to pass to the output. This is a dual-path architecture — a deliberate trade-off of area for simplicity and speed, avoiding any mode-switching logic or state machine. It is particularly well-suited for FPGAs where LUT resources are plentiful.

The decryption path is fed the output of the encryption path (cipher), so the full chain is: plaintext → Encrypt → ciphertext → Decrypt → recovered plaintext, all happening in one continuous combinational + clocked pipeline.

2.3 Module Hierarchy

Module	Inputs	Outputs
AES (Top)	in[127:0], key[127:0], mode, Clk	out[127:0]
Encryption	in[127:0], key[127:0], Clk	cipher[127:0]
Decryption_new	cipher[127:0], key[127:0], Clk	plain[127:0]
Key_expension	key[127:0], Clk	w4_7 ... w40_43 (10×128-bit round keys)
Secret_key	key[127:0], Clk	key0...key15 (16×8-bit registered bytes)
pre_state_matrix	in[127:0], key[127:0]	psm_out[127:0] (XOR result)
SubBytes	in[127:0]	SubBytes_out[127:0]
S_box	in[7:0]	out[7:0] (256-entry LUT)
ShiftRows	in[127:0]	shift_out[127:0]
MixColumns	in[127:0], Clk	MixColumns_out[127:0]
GF_mul	a[7:0], sel[1:0], Clk	out[7:0]
Inv_SubBytes	in[127:0]	SubBytes_out[127:0]
Inv_S_box	in[7:0]	out[7:0] (256-entry inverse LUT)
Inv_ShiftRows	in[127:0]	inv_shift_out[127:0]
Inv_MixColumns	in[127:0], Clk	MixColumns_out[127:0]
Inv_GF_mul	a[7:0], sel[1:0], Clk	out[7:0]

3. Module-by-Module Analysis

3.1 Secret_key — Key Register Bank

File: Secret_key.v

Purpose: This module is the first stage of the key scheduling pipeline. It receives the full 128-bit key on a single bus and, on every rising clock edge, samples and registers each byte into 16 individual 8-bit output registers (key0 through key15).

Why it exists: The Key_expansion module needs to access individual bytes of the key to perform RotWord, SubWord, and RCON XOR operations. Breaking the 128-bit bus into 16 named byte registers makes the combinational logic in Key_expansion cleaner and directly mappable to the AES specification's word-based key schedule.

Logic type: Pure sequential (registered). All 16 outputs are updated synchronously on posedge Clk. This introduces exactly one clock cycle of latency before the key bytes are available to downstream combinational logic.

```
always @(posedge Clk) begin
    key0 <= key[127:120]; // MSB byte
    ...
    key15 <= key[7:0];    // LSB byte
end
```

3.2 S_box — AES Forward Substitution Box

File: S_box.v

Purpose: The S-Box (Substitution Box) is the only non-linear element in AES. It performs a byte substitution: every 8-bit input value is replaced with a different 8-bit output value as defined by the AES specification. This non-linearity is the primary source of confusion in AES — without it, AES would be a linear transform and trivially breakable.

Mathematical background: The AES S-Box transformation has two mathematical steps — first, find the multiplicative inverse of the byte in $GF(2^8)$ (the finite field with 256 elements under the irreducible polynomial $x^8+x^4+x^3+x+1$), then apply a specific affine transformation over $GF(2)$. Rather than computing this on-the-fly (which would require complex GF arithmetic circuits), this implementation uses a pre-computed 256-entry lookup table — the standard and most area-efficient approach for FPGA targets.

Implementation: The module is a purely combinational always @(*) block with a case statement mapping all 256 possible 8-bit input values to their corresponding S-Box output. For example:

```
S_box(0x00) → 0x63   S_box(0x01) → 0x7C   S_box(0xFF) → 0x16
```

On an FPGA, Vivado synthesizes this as a set of LUTs (Look-Up Tables), which is extremely efficient. On an ASIC, it synthesizes as ROM-based or optimized logic.

Inputs: in[7:0] — one byte | **Outputs:** out[7:0] — substituted byte | **Logic:** Combinational only (no clock).

3.3 SubBytes — Parallel 16-Byte Substitution

File: SubBytes.v

Purpose: SubBytes applies the S-Box transformation to all 16 bytes of the 128-bit state simultaneously. It is the vectorized form of S_box operating on the full AES state.

Implementation: The 128-bit input is sliced into 16 individual 8-bit wires (pre_state0 through pre_state15). Then, 16 instances of S_box are instantiated in parallel — one per byte. Because all 16 S-Box instances are combinational, SubBytes itself is entirely combinational and adds zero clock cycles of latency. The 16 substituted bytes are then concatenated back into a single 128-bit output.

```
assign pre_state0 = in[127:120]; // Byte 0
S_box n1(sub0, pre_state0);      // Substitute byte 0
...
assign SubBytes_out = {sub0, sub1, ..., sub15};
```

Key engineering insight: Instantiating 16 S-Box modules in parallel means all 16 byte substitutions happen in the same clock cycle simultaneously — true hardware parallelism that software cannot replicate without SIMD instructions.

3.4 ShiftRows — Byte-Level Row Permutation

File: ShiftRows.v

Purpose: ShiftRows is a purely structural (wiring) operation. It rearranges the 16 bytes of the state according to a fixed permutation that corresponds to cyclically left-shifting each row of the 4×4 state matrix by a different offset.

Row shift amounts:

- Row 0: No shift — bytes {0, 1, 2, 3} stay in place
- Row 1: Left shift by 1 — bytes {4,5,6,7} → {5,6,7,4}
- Row 2: Left shift by 2 — bytes {8,9,10,11} → {10,11,8,9}
- Row 3: Left shift by 3 — bytes {12,13,14,15} → {15,12,13,14}

From the code, the exact wire reassignment is:

```
assign shift0=sub0;  assign shift1=sub5;  assign shift2=sub10; assign
shift3=sub15;
assign shift4=sub4;  assign shift5=sub9;  assign shift6=sub14; assign
shift7=sub3;
assign shift8=sub8;  assign shift9=sub13; assign shift10=sub2; assign
shift11=sub7;
assign shift12=sub12; assign shift13=sub1; assign shift14=sub6; assign
shift15=sub11;
```

Implementation: This module contains zero logic gates. It is purely a re-routing of wires. Vivado synthesizes this as direct wire connections with no LUT or flip-flop usage. This makes ShiftRows the most area-efficient module in the design — free in hardware.

Why it matters: ShiftRows provides inter-column diffusion. After SubBytes, all bytes within a column are related. ShiftRows mixes bytes across columns, ensuring that MixColumns in the next step operates on bytes that came from different original columns — greatly amplifying diffusion after just a few rounds.

3.5 GF_mul — Galois Field Multiplier (Forward)

File: GF_mul.v

Purpose: This is the fundamental arithmetic building block for MixColumns. It performs multiplication of an 8-bit byte by small constants (1, 2, or 3) in the Galois Field $GF(2^8)$ under the AES irreducible polynomial $p(x) = x^8 + x^4 + x^3 + x + 1$ (hex: 0x11B).

The sel[1:0] signal selects the multiplication factor:

- sel=2'b01 → output = $a \times 1 = a$ (identity)
- sel=2'b10 → output = $a \times 2 = \text{xtime}(a)$ — shift left by 1, XOR 0x1B if MSB was 1
- sel=2'b11 → output = $a \times 3 = \text{xtime}(a) \text{ XOR } a$ (since $3 = 2 + 1$ in GF)
- sel=2'b00 → output = 0x00 (unused/zero case)

The $GF(2^8)$ multiplication by 2 is implemented as:

```
if (a[7] == 1) temp <= (a << 1) ^ 8'h1B; // Reduce mod p(x)
else          temp <= a << 1;           // Simple left shift
```

Logic type: Sequential (registered on posedge Clk). Both the xtime computation and the final mux are clocked. This means GF_mul introduces 1 clock cycle of latency per use. Since MixColumns instantiates 64 GF_mul modules (16 per column group $\times$ 4 column groups), all 64 operate in parallel on the same clock edge.

3.6 MixColumns — Column Mixing Transformation

File: MixColumns.v

Purpose: MixColumns is the primary diffusion step in AES. It treats each 4-byte column of the state as a polynomial over $GF(2^8)$ and multiplies it by a fixed MDS (Maximum Distance Separable) matrix. Each output byte depends on all 4 input bytes of its column, ensuring that a single-byte change in input produces a 4-byte change in output — the so-called avalanche effect.

The AES MixColumns matrix multiplication is:

```
[2 3 1 1] [b0] [out0]
[1 2 3 1] × [b1] = [out1]      (all arithmetic in GF(2^8))
[1 1 2 3] [b2] [out2]
[3 1 1 2] [b3] [out3]
```

Implementation: The 128-bit input is divided into four groups of 4 bytes (columns). For each column, 16 instances of GF_mul are instantiated — 4 multiplications per output byte $\times$ 4 output bytes. The XOR of the four GF products gives each output byte. Total: 64 GF_mul instances across 4 column groups, all operating in parallel.

```
// Column 0, output byte 0:
GF_mul n1(m1, shift0, 2'b10, Clk); // shift0 × 2
GF_mul n2(m2, shift1, 2'b11, Clk); // shift1 × 3
GF_mul n3(m3, shift2, 2'b01, Clk); // shift2 × 1
GF_mul n4(m4, shift3, 2'b01, Clk); // shift3 × 1
assign column0 = m1 ^ m2 ^ m3 ^ m4;
```

Logic type: Mixed — GF_mul is clocked (sequential), while the final XOR assignments are combinational. Net latency: 1 clock cycle per MixColumns stage.

3.7 pre_state_matrix — AddRoundKey

File: pre_state_matrix.v

Purpose: This is the AddRoundKey operation of AES. Despite its name, pre_state_matrix performs a simple 128-bit XOR between the current state and the round key. This is the only operation in AES that directly introduces the secret key material into the data path.

```
assign psm_out = in ^ key;
```

Implementation: One line of Verilog — a 128-bit bitwise XOR. This synthesizes to exactly 128 XOR gates in hardware, one per bit. No clock, no registers — purely combinational with near-zero propagation delay.

Why XOR? In GF(2), addition and subtraction are the same operation (XOR). XOR is its own inverse: $A \oplus K \oplus K = A$. This means the same operation works for both encrypting (adding the key) and decrypting (removing the key), which is why the same pre_state_matrix module is reused in both the Encryption and Decryption_new modules.

Usage in the design: pre_state_matrix is instantiated 11 times in Encryption (initial round + 10 rounds) and 11 times in Decryption, using different round keys each time. It is the most instantiated module in the design.

3.8 Key_expension — AES-128 Key Schedule

File: Key_expension.v

Purpose: The Key Schedule (Key Expansion) takes the 128-bit original key and derives 10 additional 128-bit round keys (one for each AES round), plus the initial key itself. In total, AES-128 uses 11 round keys (the original key + 10 derived keys = 1408 bits).

Inputs: key[127:0], Clk | Outputs: w4_7, w8_11, w12_15, w16_19, w20_23, w24_27, w28_31, w32_35, w36_39, w40_43 (ten 128-bit round key buses)

RCON values hardwired in this design:

```
rcon_1=0x01  rcon_2=0x02  rcon_3=0x04  rcon_4=0x08  rcon_5=0x10  
rcon_6=0x20  rcon_7=0x40  rcon_8=0x80  rcon_9=0x1B  rcon_10=0x36
```

These are the AES-standard RCON values — powers of 2 in GF(2⁸) — used to introduce round-dependent asymmetry and prevent related-key attacks.

G-function (RotWord + SubWord + RCON XOR): For each round, the last word of the previous round key undergoes the G-function:

- RotWord: Rotate the 4-byte word left by 1 byte. In the code: assign b0=key13; b1=key14; b2=key15; b3=key12; — this reorders the bytes to achieve rotation.
- SubWord: Apply S_box to each of the 4 rotated bytes independently.
- RCON XOR: XOR the first byte of the substituted word with the round constant.

Word generation: Each new word W_i is XORed with the word four positions earlier (W_{i-4}). This creates a chain of dependencies: round 1 depends on the original key, round 2 depends on round 1, and so on.

Architecture: Entirely combinational (except Secret_key which registers the initial key bytes). All 10 round keys are computed simultaneously using parallel combinational logic — a fully unrolled key schedule. This is the most area-intensive approach but gives single-cycle key availability.

3.9 Inv_S_box & Inv_SubBytes — Inverse Substitution

Files: Inv_S_box.v, Inv_SubBytes.v

The inverse S-Box is a separate 256-entry LUT that exactly reverses the forward S-Box transformation. For decryption, each byte that was substituted during encryption must be restored to its pre-substitution value. The structure mirrors SubBytes exactly — 16 parallel Inv_S_box instances operating combinationally on the 128-bit state.

```
Inv_S_box: 0x00→0x52, 0x63→0x00, 0x7C→0x01 ... (complete 256-entry inverse table)
```

3.10 Inv_ShiftRows — Inverse Row Shift

File: Inv_ShiftRows.v

InvShiftRows reverses the left-shift permutation of ShiftRows by right-shifting rows instead. Like ShiftRows, it is purely a wire re-routing — no gates, no registers, zero area cost. The right-shift permutation is:

- Row 0: No shift — bytes {0,1,2,3} stay
- Row 1: Right shift by 1 — byte positions {4,5,6,7} reassigned
- Row 2: Right shift by 2
- Row 3: Right shift by 3

```
assign shift0=cipher0;  assign shift1=cipher13; assign shift2=cipher10;
assign shift3=cipher7;

assign shift4=cipher4;  assign shift5=cipher1;  assign shift6=cipher14;
assign shift7=cipher11;
```

3.11 Inv_GF_mul & Inv_MixColumns — Inverse Column Mixing

Files: Inv_GF_mul.v, Inv_MixColumns.v

Inv_MixColumns reverses the MixColumns operation using the inverse MDS matrix. The inverse matrix uses multiplications by 0x09, 0x0B, 0x0D, and 0x0E instead of 1, 2, and 3.

Inv_GF_mul computes these higher-order multiplications by chaining xtime operations:

```
mul2 = xtime(a)           // × 2
mul4 = xtime(mul2)        // × 4
mul8 = xtime(mul4)        // × 8
sel=00 → mul8^mul4^mul2    // × 0E (14)
sel=01 → mul8^mul2^a       // × 0B (11)
sel=10 → mul8^mul4^a       // × 0D (13)
sel=11 → mul8^a            // × 09 (9)
```

Key difference from GF_mul: Inv_GF_mul uses an always @(*) combinational block for the intermediate multiplications (mul2, mul4, mul8), and only the final output is clocked. This makes it more combinational efficient than the forward GF_mul. The inverse MixColumns matrix:

```
[0E 0B 0D 09]
[09 0E 0B 0D]
[0D 09 0E 0B]
[0B 0D 09 0E]
```


4. AES Encryption Flow — Round by Round

4.1 Initial Round (Round 0)

Before any of the 10 AES rounds begin, the plaintext is XORed with the original 128-bit key. In the code:

```
pre_state_matrix n1(psm_out1, in, key);    // AddRoundKey with original key
```

This initial whitening step ensures that an attacker cannot simply observe the first round's SubBytes output to learn information about the plaintext.

4.2 Rounds 1–9 (Main Rounds)

Each of rounds 1 through 9 applies the four AES transformations in sequence:

Step 1 — SubBytes: Each of the 16 state bytes is independently substituted through the S-Box. Provides non-linearity and confusion.

```
SubBytes n2(SubBytes_out1, psm_out1);
```

Step 2 — ShiftRows: The 16 state bytes are rearranged by rotating rows. Provides inter-column diffusion.

```
ShiftRows n3(shift_out1, SubBytes_out1);
```

Step 3 — MixColumns: Each 4-byte column is multiplied by the MDS matrix in $GF(2^8)$. Provides intra-column diffusion and ensures full avalanche after multiple rounds.

```
MixColumns n4(MixColumns_out1, shift_out1, Clk);
```

Step 4 — AddRoundKey: The state is XORed with the round-specific key derived by Key_expansion.

```
pre_state_matrix n5(psm_out2, MixColumns_out1, w4_7);    // Round 1 key
pre_state_matrix n9(psm_out3, MixColumns_out2, w8_11);    // Round 2 key
... and so on through w36_39 for round 9 ...
```

4.3 Final Round (Round 10)

The final round omits MixColumns — this is specified in the AES standard and ensures the cipher is invertible without any special-casing in decryption. The sequence is SubBytes → ShiftRows → AddRoundKey only:

```
SubBytes n38(SubBytes_out10, psm_out10);
ShiftRows n39(shift_out10, SubBytes_out10);
// No MixColumns here!
pre_state_matrix n40(psm_out11, shift_out10, w40_43);    // Final round key
assign cipher = psm_out11;
```

4.4 AES Decryption Flow

The Decryption_new module reverses all 10 rounds. The decryption order applies inverse operations in reverse sequence, using round keys in reverse order (w40_43 first, original key last):

Initial Round (Decryption): AddRoundKey with last round key (w40_43):

```
pre_state_matrix n1(psm_out1, cipher, w40_43);
```

Rounds 1–9 (Decryption): For each round: Inv_ShiftRows → Inv_SubBytes → AddRoundKey → Inv_MixColumns

```
Inv_ShiftRows n2(InvShiftRows_out1, psm_out1);
```

```
Inv_SubBytes n3(InvSubBytes_out1, InvShiftRows_out1);
```

```
pre_state_matrix n4(psm_out2, InvSubBytes_out1, w36_39); // Round key 9
```

```
Inv_MixColumns n5(InvMixColumns_out1, psm_out2, Clk);
```

Final Round (Decryption): Inv_ShiftRows → Inv_SubBytes → AddRoundKey with original key (no Inv_MixColumns):

```
Inv_ShiftRows n38(InvShiftRows_out10, InvMixColumns_out9);
```

```
Inv_SubBytes n39(InvSubBytes_out10, InvShiftRows_out10);
```

```
pre_state_matrix n40(psm_out11, InvSubBytes_out10, key);
```

```
assign plain = psm_out11;
```

5. Implementation Details

5.1 Design Style: Fully Unrolled Architecture

This design uses a fully unrolled, non-iterative architecture. Instead of sharing a single round datapath and using a counter to iterate 10 times (iterative architecture), all 10 rounds are physically instantiated as separate hardware in parallel chains. This means:

- All 10 encryption rounds and 10 decryption rounds exist simultaneously as unique hardware instances.
- The design is deeply pipelined in the sense that all round modules are always active — data flows through the chain combinational and clocked stages.
- Higher area usage compared to an iterative design, but much simpler control logic (no FSM needed).
- The entire encryption or decryption result is available after the propagation delay of 10 chained round stages plus clock latency from GF_mul and Secret_key registers.

5.2 Clock & Sequential Elements

Only three module types introduce clock-dependent (sequential) behavior:

- Secret_key: Registers all 16 key bytes on posedge Clk. Introduces 1 cycle latency before key bytes are valid.
- GF_mul (forward): Both the xtime computation and the output mux are registered on posedge Clk. Each MixColumns stage requires 1 clock cycle.
- Inv_GF_mul (inverse): Only the final output is registered. Intermediate multiplications (mul2, mul4, mul8) are combinational.

All other modules (SubBytes, ShiftRows, AddRoundKey/pre_state_matrix, S_box, Key_expansion word XORs) are purely combinational — they respond immediately to input changes with no clock dependency.

5.3 Wires and Interconnect

The design makes heavy use of named wire buses to connect module stages. In Encryption.v alone: 11 pre_state_matrix output wires (psm_out1...psm_out11), 10 SubBytes output wires, 10 ShiftRows output wires, and 9 MixColumns output wires — totaling $40 \times 128\text{-bit} = 5,120\text{-bit}$ wire buses within just the encryption path.

5.4 Key Design Decisions & Trade-offs

Dual-path (Enc+Dec always active)	Simpler control, but 2× area vs. single shared path
Fully unrolled rounds	No FSM needed, high throughput, but very high gate count
LUT-based S-Box (case statement)	Fast combinational lookup, simple to verify, FPGA-optimal
GF_mul clocked (registered output)	Avoids long combinational chains but adds 1-cycle latency per round
Hardwired RCON constants	Zero area overhead, constants never change in AES-128
pre_state_matrix as AddRoundKey	Clean separation of concerns, easily reusable for both enc/dec
Wire slicing for byte extraction	Avoids explicit byte arrays, directly synthesizable, readable

6. Waveform Observations (Vivado Simulation)

Note: The waveform screenshot was not provided at report generation time. The following describes the expected simulation behavior based on the RTL architecture. Upload your waveform image to have this section updated with actual observed values.

6.1 Expected Signal Behavior

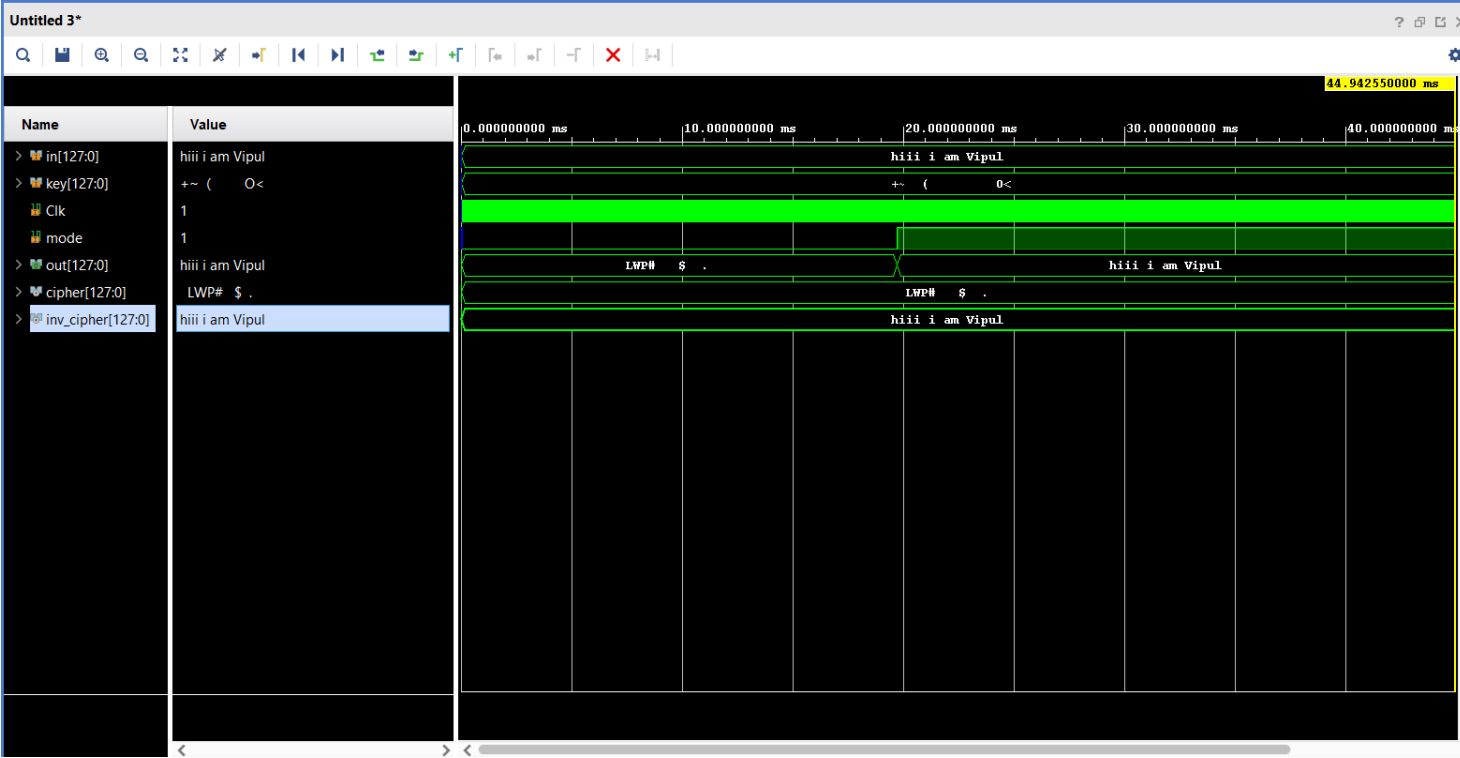
- Clk: Periodic clock toggling. All registered outputs (key0-key15, GF_mul outputs, Inv_GF_mul outputs) update on the rising edge.
- in[127:0] & key[127:0]: Applied as static inputs before simulation starts. In testbench fashion, these are typically assigned at time 0 or after a reset.
- mode: When mode=0, out = cipher (encryption result). When mode=1, out = inv_cipher (decryption result).
- cipher[127:0]: Should become valid after the combinational propagation delay through the 10-round encryption chain + clock latency from GF_mul registers. In simulation, expect the value to stabilize 1-2 clock cycles after inputs are applied.
- plain[127:0]: Should match the original in[127:0] when decryption is applied to cipher[127:0] — this is the primary functional verification check.

6.2 Verification Checklist

- After 1 clock edge: key0-key15 should equal the 16 bytes of the input key (Secret_key registration).
- After propagation: cipher $\neq$ in (encryption has occurred — output looks random).
- Functional check: plain == in when mode=1 (full encrypt-then-decrypt round trip).
- Changing mode from 0→1: out transitions from cipher to plain without any additional clock cycles (combinational mux in always @(*)).
- S-Box verification: Apply known NIST test vector (e.g., in=0x00112233..., key=0x000102...) and verify cipher matches 0x69C4E0D8...

I changed the radix of my input
actually my input is :
68696969206920616d20566970756c20
Which is “hii i am vipul “ when I change radix to ASCII

So,
input- 68696969206920616d20566970756c20 in ASCII (hii i am vipul)
key- 2b7e151628aed2a6abf7158809cf4f3c



7. Optimization & Future Improvements

7.1 Pipelined Architecture

The current design has GF_mul and Secret_key as the only registered stages, with long combinational paths through the S-Box chain. A fully pipelined architecture would insert pipeline registers (flip-flops) between every round stage. This would allow a new plaintext to enter the pipeline every clock cycle (throughput = 128 bits/cycle) while the previous block is still being processed in later stages — dramatically improving throughput for bulk encryption.

7.2 Iterative Architecture for Area Reduction

The current fully unrolled design instantiates 10 sets of SubBytes/ShiftRows/MixColumns/AddRoundKey for each of encryption and decryption. An iterative design would use a single shared round datapath, a 4-bit counter to track the current round, and a multiplexer to select the correct round key. This would reduce area by approximately 10×, at the cost of requiring 10+ clock cycles to complete one block (latency vs. throughput trade-off).

7.3 FPGA Resource Optimization

- Block RAM S-Box: Replace the case-statement S-Box with FPGA Block RAM (BRAM). Vivado can infer BRAM from ROM-style case statements, but explicit BRAM instantiation gives better control over placement and timing.
- DSP Slices for GF_mul: Although GF multiplication is over $GF(2^8)$ (XOR-based, no true integer multiply), modern FPGAs have DSP48E2 slices that can be repurposed for finite field arithmetic with careful mapping.
- Carry-chain XOR: The 128-bit XOR in pre_state_matrix can be explicitly mapped to FPGA carry chains for minimal propagation delay.

7.4 Side-Channel Attack Protection

Hardware AES implementations are vulnerable to side-channel attacks — an adversary can observe power consumption, electromagnetic emissions, or timing variations to extract the secret key without breaking the mathematical structure. Mitigations include:

- Masking: XOR all intermediate state values with random masks before S-Box lookup, removing the correlation between data values and power consumption.
- Constant-time S-Box: Ensure the S-Box lookup takes identical time regardless of input — already achieved by the case-statement LUT approach in this design.
- Power balancing: Dual-rail logic or other balanced logic families to equalize power consumption regardless of data values.

7.5 AES Modes of Operation

The current design implements raw AES (ECB mode — Electronic Code Book). Real systems use AES in chained modes for enhanced security:

- CBC (Cipher Block Chaining): XOR each plaintext block with the previous ciphertext block before encryption. Prevents identical plaintext blocks from producing identical ciphertext.
- CTR (Counter Mode): Encrypt a counter value and XOR with plaintext — converts AES block cipher into a stream cipher. Allows parallelization of encryption.
- GCM (Galois/Counter Mode): Adds authenticated encryption (integrity + confidentiality) — the most widely deployed AES mode in TLS 1.3.

7.6 AXI4 Interface for SoC Integration

For FPGA deployment in a real system (e.g., Zynq SoC), the AES core would be wrapped with an AXI4-Lite or AXI4-Stream interface. This allows the ARM processor on the Zynq PS to communicate with the AES PL (FPGA fabric) accelerator using standard memory-mapped registers.

9. References

- NIST FIPS PUB 197 — Advanced Encryption Standard (AES), November 2001

<https://nsarchive.gwu.edu/themes/custom/nsarchive/templates/pdfs/web/viewer.html?file=https%3A%2F%2Fnsarchive.gwu.edu%2Fsites%2Fdefault%2Ffiles%2Fdocuments%2F5989783%2FNational-Security-Archive-National-Institute-of.pdf>

- NIST SP 800-38A — Recommendation for Block Cipher Modes of Operation
- Special thanks to my batchmate and my close friend Yashi Sharma.
- Xilinx Vivado Design Suite User Guide — Synthesis (UG901)
- FIPS 197 S-Box construction — $GF(2^8)$ multiplicative inverse + affine transform
- Daemen, J. & Rijmen, V. — The Design of Rijndael: AES – The Advanced Encryption Standard (Springer)
-
- GitHub Repository: github.com/vipul3112/AES-128-bit
- NESO Academy
<https://www.youtube.com/watch?v=3MPkc-PFSRI>